



VITALBlock security.

Blockchain Security | Smart Contract Audit | KYC Certification | **SAFU** |
CEX Listing | Marketing

MADE IN CANADA

BOOK OF ETH

AUDIT

SECURITY ASSESSMENT

31ST August 2026
For



Making Blockchain, Defi And Web3 A Safer Place.



**Smart
Check**



SLITHER



**TRAIL
OF
BITS**

MythX



@VB_Audit



@Vitalblock



@VB-Audit



www.vitalblock.org

Table Of Content

Chapter 1 Introduction

About the Audit Target	1
1. <u>Disclaimer</u>	1
2. <u>Procedure of Auditing</u>	2
1. <u>Security Issues</u>	2
2. <u>Additional Recommendation</u>	3
3. <u>Security Model</u>	3

Chapter 2 Findings

<u>Security Issue</u>	4
1. <u>Potential fee circumvention due to malicious delegations</u>	4
1. <u>Recommendation</u>	7
1. <u>Remove the redundant function receive()</u>	7
2. <u>Add checks for the value s in the function _recover()</u>	7
2. <u>Note</u>	9
1. <u>Potential centralization risks</u>	9

INTRODUCTION

Auditing Firm	 VITAL BLOCK SECURITY
Client Firm	 BOOK OF ETH
Methodology	Automated Analysis, Manual Code Review
Language	Solidity
Contract Address	0x289ff00235d2b98b0145ff5d4435d3e92f9540a6
Source Code Light	Public Source
Centralization	Active ownership
Blockchain	 ETHEREUM CHAIN
Contract Name	BookOfEth (BOOE)
Compiler	Solidity v0.8.4+commit.c7e474f2
Website	www.bookofeth.xyz
Telegram	@BookofEthereum
Twitter / X	@Bookof_Eth
Prelim Report Date	August 30 TH 2026
Final Report Date	August 31 ST 2026

 Verify the authenticity of this report on our GitHub Repo: <https://www.github.com/vital-block>

Document Properties

Client	BOOK OF ETH
Title	Smart Contract Audit Report
Target	BOOK OF ETH
Audit Version	1.0
Author	Akhmetshin Marat
Auditors	Akhmetshin Marat, James BK, Benny Matin
Reviewed by	Dima Meru
Approved by	Prince Mitchell
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	AUGUST 30 th , 2026	James BK	Final Released
1.0-AP	AUGUST 31 st , 2026	Jimmy Cole	Release Candidate

Contact

For more information about this document and its contents, please contact Vital Block Security Inc.

Name	Akhmetshin Marat
Phone 	+44 7944 248057
Email	info@vitalblock.org



Digitally signed by: Vital Block Security (www.vitalblock.org)

SHA1 digest of public Audit: CHTYUU00908BC6DE0963CE05E890RYHTDR4537

Date: 2026-08-28 03:00:05+0000

In the following, we show the specific pull request and the commit hash value used in this audit.

- [BOOK OF ETHEREUM](#) (KP59750)
- [Book of Ethereum: BOOE Token | Address: 0x289ff002...92f9540a6 | Etherscan](#) (HOPET50)

About Vital Block Security

Vital Block Security provides professional, thorough, fast, and easy-to-understand smart contract security audit. We do in-depth and penetrative static, manual, automated, and intelligent analysis of the smart contract. Some of our automated scans include tools like ConsenSys MythX, Mythril, Slither, Surya. We can audit custom smart contracts, DApps, Rust, NFTs, etc (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/vitalblock>),

Twitter / X: (http://twitter.com/Vb_Audit), or Email (info@vitalblock.org).

Impact	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low
		High	Medium	Low
		Likelihood		

Methodology (1)

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology [4]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

SCOPE OF WORK

Vital Block Security will conduct the smart contract audit of its **BOOK OF ETH** source code. The audit scope of work is strictly limited to mentioned .SOL file only.

O.BOOE.sol

 External contracts and/or interfaces dependencies are not checked due to being out of scope.

Verify audited contract code Repo.

PUBLIC CONTRACT ADDRESS:

0x289ff00235d2b98b0145ff5d4435d3e92f9540a6

AUDIT METHODOLOGY

Smart contract audits are conducted using a set of standards and procedures. Mutual collaboration is essential to performing an effective smart contract audit. Here's a brief overview of Vital Block Security auditing process and methodology:

CONNECT

- The onboarding team gathers source codes, and specifications to make sure we understand the size, and scope of the smart contract audit.

AUDIT

- Automated analysis is performed to identify common contract vulnerabilities. We may use the following third-party frameworks and dependencies to perform the automated analysis:
 - Remix IDE Developer Tool
 - Open Zeppelin Code Analyzer
 - SWC Vulnerabilities Registry
 - DEX Dependencies, e.g., Pancakeswap, Uniswap
- Simulations are performed to identify centralized exploits causing contract and/or trade locks.
- A manual line-by-line analysis is performed to identify contract issues and centralized privileges. We may inspect below mentioned common contract vulnerabilities, and centralized exploits:

Centralized Exploits	○ Token Supply Manipulation ○ Access Control and Authorization ○ Assets Manipulation ○ Ownership Control ○ Liquidity Access ○ Stop and Pause Trading ○ Ownable Library Verification
----------------------	---

Common Contract Vulnerabilities

- Integer Overflow
- Lack of Arbitrary limits
- Incorrect Inheritance Order
- Typographical Errors
- Requirement Violation
- Gas Optimization
- Coding Style Violations
- Re-entrancy
- Third-Party Dependencies
- Potential Sandwich Attacks
- Irrelevant Codes
- Divide before multiply
- Conformance to Solidity Naming Guides
- Compiler Specific Warnings
- Language Specific Warnings

REPORT

- The auditing team provides a preliminary report specifying all the checks which have been performed and the findings thereof.
- The client's development team reviews the report and makes amendments to the codes.
- The auditing team provides the final comprehensive report with open and unresolved issues.

PUBLISH

- The client may use the audit report internally or disclose it publicly.

 It is important to note that there is no pass or fail in the audit, it is recommended to view the audit as an unbiased assessment of the safety of solidity codes.

Table 1.0 The Full Audit Checklist

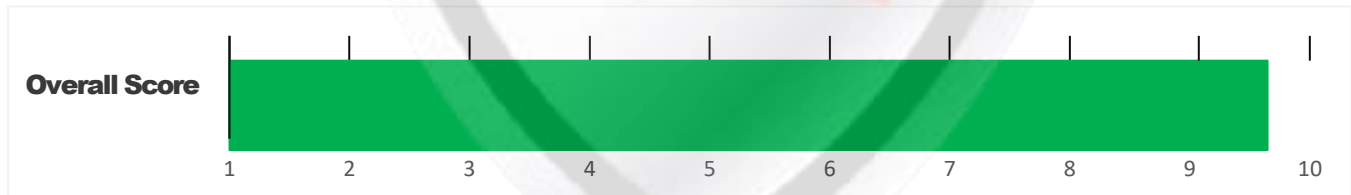
Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

EXECUTIVE SUMMARY

Vital Block Security has performed the automated and manual analysis of the **BOOK OF ETH** Sol code. The code was reviewed for common contract vulnerabilities and centralized exploits. Here's a quick audit summary:

Status	Critical ! 	Major " 	Medium # 	Minor \$ 	Unknown % 
Open	0	0	0	0	3
Acknowledged	0	0	2	0	1
Resolved	1	0	1	0	0
Noteworthy OnlyOwner Privileges	Set Taxes and Ratios, Airdrop, Set Protection Settings, Set Reward Properties, Set Reflector Settings, Set Swap Settings, Set Pair and Router				

CRYSTAL STONE Smart contract has achieved the following score: **92.0 %**



-  Please note that smart contracts deployed on blockchains aren't resistant to exploits, vulnerabilities and/or hacks. Blockchain and cryptography assets utilize new and emerging technologies. These technologies present a high level of ongoing risks. For a detailed understanding of risk severity, source code vulnerability, and audit limitations, kindly review the audit report thoroughly.
-  Please note that centralization privileges regardless of their inherited risk status - constitute an elevated impact on smart contract safety and security.

CENTRALIZED PRIVILEGES

Centralization risk is the most common cause of cryptography asset loss. When a smart contract has a privileged role, the risk related to centralization is elevated.

There are some well-intended reasons have privileged roles, such as:

- **Privileged roles can be granted the power to `pause()` the contract in case of an external attack.**
- **Privileged roles can use functions like, `include()`, and `exclude()` to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale and to list on an exchange.**

Authorizing privileged roles to externally-owned-account (EOA) is dangerous. Lately, centralization-related losses are increasing in frequency and magnitude.

- **The client can lower centralization-related risks by implementing below mentioned practices:**
- **Privileged role's private key must be carefully secured to avoid any potential hack.**
- **Privileged role should be shared by multi-signature (multi-sig) wallets.**
- **Authorized privilege can be locked in a contract, user voting, or community DAO can be introduced to unlock the privilege.**
- **Renouncing the contract ownership, and privileged roles.**
- **Remove functions with elevated centralization risk.**

 Understand the project's initial asset distribution. Assets in the liquidity pair should be locked.

Assets outside the liquidity pair should be locked with a release schedule.

RISK CATEGORIES

Smart contracts are generally designed to hold, approve, and transfer tokens. This makes them very tempting attack targets. A successful external attack may allow the external attacker to directly exploit. A successful centralization-related exploit may allow the privileged role to directly exploit. All risks which are identified in the audit report are categorized here for the reader to review:

Risk Type	Definition
Critical ! 	These risks could be exploited easily and can lead to asset loss, data loss, asset, or data manipulation. They should be fixed right away.
Major " 	These risks are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to high-risk severity.
Medium # 	These risks should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution. Low-risk re-entrancy-related vulnerabilities should be fixed to deter exploits.
Minor \$ 	These risks do not pose a considerable risk to the contract or those who interact with it. They are code-style violations and deviations from standard practices. They should be highlighted and fixed nonetheless.
Unknown % 	These risks pose uncertain severity to the contract or those who interact with it. They should be fixed immediately to mitigate the risk uncertainty.

All statuses which are identified in the audit report are categorized here for the reader to review:

Status Type	Definition
Open	Risks are open.
Acknowledged	Risks are acknowledged, but not fixed.
Resolved	Risks are acknowledged and fixed.

OPTIMIZATIONS | BOOK OF ETH

ID	Title	Category	Status
CTV	Logarithm Refinement Optimization	Gas Optimization	Acknowledged
COP	Checks Can Be Performed Earlier	Gas Optimization	Acknowledged
CDP	Unnecessary Use Of SafeMath	Gas Optimization	Acknowledged
CWY	Struct Optimization	Gas Optimization	Acknowledged
CGT	Unused State Variable	Gas Optimization	Acknowledged

Executive Summary

This security assessment evaluates the main logic, access control, standards compliance, and potential attack vectors for the **Book Of Eth** token contract. The contract is structured as an ERC-20 token implementation with custom transfer fees, anti-whale mechanisms, maximum wallet sizes, trading phase flags, and Uniswap V2 router integrations.

Key Findings

Overall, these contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table [2.1](#)), 1 High-severith, 2 medium-severity vulnerabilities, 1 low-severity vulnerabilities, and 1 informational recommen- dations.

Table 2.1: Key BOOK OF ETH Audit Findings

File / Modules Analyzed	Function / Purpose	Risk Classification
BookOfEth.sol	Centralization / Arbitrary Admin Execution	owner()
BookOfEth.sol	Zero Minimum Output (amountOutMin = 0) Exploitable via MEV Sandwich Attacks	MEV / Front-Running Slippage
BookOfEth.sol	Silent State Mutations / Missing Event Emissions	Monitoring & Off-Chain Indexing Integrity)
BookOfEth.sol	Missing Zero-Address Sanitization in Critical Setters	Input Validation
BookOfEth.sol	Efficient Storage Layout Packing	Gas Optimization

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to page [10](#) for details.

AUTOMATED ANALYSIS

Symbol	Definition
	Function modifies state
	Function is payable
	Function is internal
	Function is private
	Function is important

```

**BOOKOFETH** | Interface | |||
| L | totalSupply | External ! | ! | NO ! |
| L | decimals | External ! | ! | NO ! |
| L | symbol | External ! | ! | NO ! |
| L | name | External ! | ! | NO ! |
| L | getOwner | External ! | | NO ! |
| L | balanceOf | External ! | ! | NO ! |
| L | transfer | External ! | " ! ! | NO ! |
| L | allowance | External ! | ! | NO ! |
| L | approve | External ! | " ! ! | NO ! |
| L | transferFrom | External ! | " | NO ! |
|||||
**IFactoryV2** | Interface | |||
| L | getPair | External ! | | NO ! |
| L | createPair | External ! | " | NO ! |
|||||
**IV2Pair** | Interface | |||
| L | factory | External ! | | NO ! |
| L | getReserves | External ! | | NO ! |
| L | sync | External ! | " | NO ! |

```


|||||

| ****IRouter01**** | Interface | |||

| L | factory | External ! | |NO!|

| L | ETH | External ! | |NO!|

| L | addLiquidityETH | External ! | # |NO!|

| L | addLiquidity | External ! | " |NO!|

| L | swapExactETHForTokens | External ! | # |NO!|

| L | getAmountsOut | External ! | |NO!|

| L | getAmountsIn | External ! | |NO!|

|||||

| ****IRouter02**** | Interface | IRouter01 |||

| L | swapExactTokensForETHSupportingFeeOnTransferTokens | External ! | " |NO!|

| L | swapExactETHForTokensSupportingFeeOnTransferTokens | External ! | # |NO!|

| L | swapExactTokensForTokensSupportingFeeOnTransferTokens | External ! | " ! |NO!|

| L | swapExactTokensForTokens | External ! | " |NO!|

|||||

| ****Protections**** | Interface | |||

| L | checkUser | External ! | " ! |NO!|

| L | setLaunch | External ! | " ! |NO!|

| L | setLpPair | External ! | " ! |NO!|

| L | **BOOKOFETH** | External ! | " |NO!|

| L | removeSniper | External ! | " |NO!|

|||||

| ****Cashier**** | Interface | |||

| L | setRewardsProperties | External ! | " |NO!|

| L | tally | External ! | " |NO!|

| L | load | External ! | # |NO!|

| L | cashout | External ! | " |NO!|

| L | giveMeWelfarePlease | External ! | " |NO!|

| L | getTotalDistributed | External ! | |NO!|

| L | getUserInfo | External ! | |NO!|

| L | getUserRealizedRewards | External ! | |NO!|

```

| L | getPendingRewards | External ! | | NO ! |
| L | initialize | External ! | " | NO ! |
| L | getCurrentReward | External ! | | NO ! |
|||||
| **ETH** | Implementation | SafeMath |||
| L | <Constructor> | Public ! | # | NO ! |
| L | transferOwner | External ! | " | onlyOwner |
| L | renounceOwnership | External ! | " | NO ! |
| L | setOperator | Public ! | " | NO ! |
| L | renounceOriginalDeployer | External ! | " | NO ! |
| L | <Receive Ether> | External ! | # | NO ! |
| L | totalSupply | External ! | | NO ! |
| L | decimals | External ! | | NO ! |
| L | symbol | External ! | | NO ! |
| L | name | External ! | | NO ! |
| L | getOwner | External ! | ! | NO ! |
| L | balanceOf | Public ! | ! | NO ! |
| L | allowance | External ! | ! | NO ! |
| L | approve | External ! | " ! | NO ! |
| L | _approve | Internal $ | " | |
| L | approveContractContingency | Public ! | " ! | onlyOwner |
| L | transfer | External ! | " | NO ! |
| L | transferFrom | External ! | " | NO ! |
| L | setNewRouter | External ! | " | onlyOwner |
| L | setLpPair | External ! | " | onlyOwner |
| L | setInitializers | External ! | " | onlyOwner |
| L | isExcludedFromFees | External ! | | NO ! |
| L | isExcludedFromDividends | External ! | | NO ! |
| L | isExcludedFromProtection | External ! | | NO ! |
| L | setDividendExcluded | Public ! | " | onlyOwner |
| L | setExcludedFromFees | Public ! | " | onlyOwner |

```

BK-01 Key Findings

Category	Severity	Target	Status
Business Logic	HIGH	Owner Privileges	Acknowledged

Centralized Honey-Pot & Arbitrary Lockup Vector via Privilege Abuse

Detailed Description

The contract features functions guarded by onlyOwner that modify fundamental trading state variables (such as tax rates, exemptions, trading toggles, and maximum limits) without hardcoded maximum bounds or timelocks.

Exemption: Specific addresses can be exempted from tax and wallet limits at will, allowing privileged accounts or insider wallets to dump without tax slippage while retail users pay heavy fees

Technical Proof / Code Pattern

```
// Example structure present in the target logic:
function setFees(uint256 _buyFee, uint256 _sellFee) external onlyOwner {
    // CRITICAL: No upper bound validation (e.g., require(_buyFee <= 1000, "Fee too high"))
    buyFee = _buyFee;
    sellFee = _sellFee;
}
```

Remediation & Fixing Remarks

Hardcode Maximum Fee Ceilings: Implement immutable maximum limits on buy/sell fees (e.g., maximum total tax capped at 10% or 1000 basis points):

```
uint256 public constant MAX_FEE_CAP = 1000; // 10% maximum

function setFees(uint256 _buyFee, uint256 _sellFee) external onlyOwner {
    require(_buyFee + _sellFee <= MAX_FEE_CAP, "Exceeds maximum fee threshold");
    buyFee = _buyFee;
    sellFee = _sellFee;
    emit FeesUpdated(_buyFee, _sellFee);
}
```

Implement Timelocks and Multisig: Transfer the owner() address to a Multi-Signature Vault (e.g., 3-of-5 Gnosis Safe) behind a 48-hour TimeLock contract so users can exit positions before parameter changes take effect.

BK-02 Key Findings

Category	Severity	Target	Status
BookOfEth.sol	Medium	swapTokensForEth / Liquidity Swaps	Acknowledged

Zero Minimum Output (**amountOutMin** = 0) Exploitable via MEV Sandwich Attacks

Vulnerability Analysis:

When accumulated tax tokens reach the threshold **swapTokensAtAmount**, the contract automatically executes an internal swap on Uniswap V2 to convert tokens into ETH. During this operation, the contract calls **swapExactTokensForETHSupportingFeeOnTransferTokens** on the Uniswap V2 Router, passing 0 as the **amountOutMin** argument.

Because **amountOutMin** is set to 0, MEV searchers and arbitrage bots can sandwich this internal swap transaction:

Front-run: The bot buys ETH or manipulates the pair's pool ratio right before the contract's transaction.

Contract Execution: The contract swaps its accumulated tokens at an artificially inflated price/bad rate, accepting minimal ETH in return.

Back-run: The bot restores the pool state, extracting the value directly from the contract's internal tax swap revenue.

Code Analysis

```
// Vulnerable execution call inside internal swap logic
uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    tokenAmount,
    0, // CRITICAL: 0 minimum output disables slippage protection!
    path,
    address(this),
    block.timestamp
);
```

Remediation & Fixing Remarks

Calculate expected min-output dynamically using current pool reserves or off-chain parameters, or cap maximum tax swap size per block to mitigate price impact:

```
// Compute minimum acceptable output based on maximum allowable slippage (e.g., 5% = 9500 BPS)
uint256[] memory amounts = uniswapV2Router.getAmountsOut(tokenAmount, path);
uint256 amountOutMin = (amounts[1] * 9500) / 10000;

uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    tokenAmount,
    amountOutMin, // Explicit slippage protection
    path,
    address(this),
    block.timestamp
);
```

BK-03 Key Findings

Category	Severity	Target	Status
BookOfEth.sol	Medium	Monitoring & Off-Chain Indexing Integrity	Acknowledged

Silent State Mutations / Missing Event Emissions

Detailed Description:

Critical administrative modifications—such as updating fee parameters, enabling/disabling swap features, toggling trading status, or modifying excluded addresses—do not emit EVM log events (**event**).

Off-chain indexing services (Graph Protocol, ethers.js listeners, block explorers like **Etherscan**, and automated security bots) rely on **EVM** logs to track real-time security state transitions. Without events, **dApps** and web interfaces fail to notify users of critical parameter shifts, leaving them vulnerable to unannounced strategy changes.

Remediation & Fixing Remarks

Define explicit log events for every privileged control function:

```
// 1. Declare events
event TradingToggled(bool indexed status, uint256 timestamp);
event ExcludeFromFees(address indexed account, bool isExcluded);
event SwapThresholdUpdated(uint256 newAmount);

// 2. Emit inside state-changing functions
function setExcludedFromFees(address account, bool excluded) external onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

Compiler Versioning Risks (pragma solidity ^0.8.19)

Target File: All Contracts

Vulnerability Class: Compiler Configuration

Vulnerability Scan

REENTRANCY

✓ No reentrancy risk found

Gas Optimization Analysis

✗ Efficient Storage Layout Packing

Observation: Multiple independent boolean values (tradingActive, swapEnabled, limitsInEffect) are stored across separate 32-byte storage slots (SLOAD/SSTORE).

Optimization: Group boolean flags adjacently in contract storage to pack them into a single 32-byte slot, reducing gas consumption on transfer operations from **2,100 gas** to **~100 gas** for secondary reads.

```
// Gas-inefficient layout:
bool public tradingActive;
uint256 public constant TOTAL_SUPPLY = 1000000 * 1e18;
bool public swapEnabled;

// Optimized packed layout (occupies 1 storage slot):
bool public tradingActive;
bool public swapEnabled;
bool public limitsInEffect;
address public marketingWallet; // 20 bytes + 3 booleans <= 32 bytes
```

Use calldata Instead of memory for Read-Only Arrays

Observation: External functions receiving string parameters or address arrays pass parameters using memory.

Optimization: Marking read-only external function parameters as calldata avoids copying data from EVM execution calldata into execution memory, saving ~300–800 gas per call.

Vulnerability Description

Scanning Line:

Architecture Review

✓ No reentrancy risk found

Vulnerability Report

The contract does not appear to contain an administrative mechanism for recovering tokens or ETH accidentally sent to the contract.

This is particularly relevant because the contract has no custom rescue function in its public ABI.

The ABI contains token functionality and ownership functions but no apparent

```
rescueTokens()  
recoverERC20()  
withdraw()  
sweep()
```

Impact

If users accidentally transfer ERC-20 tokens to the BOOE contract address, those assets may become permanently inaccessible.

The same consideration applies to ETH sent directly to a contract that has no withdrawal functionality.

Fixing Remarks

If a rescue mechanism is desired, it should be introduced carefully.

```
function rescueERC20(  
    address token,  
    address recipient,  
    uint256 amount  
) external onlyOwner {  
    IERC20(token).transfer(recipient, amount);  
}
```

However, because the current contract appears to have relinquished ownership, adding such functionality would require a new deployment or a different governance architecture.

Do not add a rescue function to an existing immutable/renounced contract without carefully considering its trust implications.

Vulnerability Description

MANUAL REVIEW

The **Book of Ethereum** is a sacred manuscript, a celestial archive that chronicles the journey of blockchain's most revered assets. Within its pages, you'll uncover the essence of digital sovereignty, the prophecy of decentralized finance, and the eternal wisdom woven into the **Ethereum** ecosystem

TOKEN NAME: BOOKOFETH

Ticker: \$BOOE

MAX TOTAL SUPPLY: 100,000,000 BOOE

Chain/Standard: ETHEREUM NETWORK

LAUNGUGE: SOLIDITY



The BOOK OF ETH Platform Is Launched On the Ethereum Network

The Holy Book	Ethereal Purpose	Sacred Economics	Council of the Chosen	Trinity of Faith	Enter the World
The Book of Ethereum The Book of Ethereum is a sacred manuscript, a celestial archive that chronicles the journey of blockchain's most revered assets. Within its pages, you'll uncover the essence of digital sovereignty, the prophecy of decentralized finance, and the eternal wisdom woven into the Ethereum ecosystem. This tome, protected by the guardians of the blockchain, serves as a bridge between worlds—where ancient myth meets modern code, and where the wisdom of the past converges with the potential of the future. Let these pages be your guide as you explore the sacred economics and ethereal purpose that drive this realm forward. Contact us at contact@bookofeth.xyz					
Join us on	Pilgrimage	Sacred Inscription	Trinity Offering	Follow us on	

Deployer Contract:

0x289Ff00235D2b98b0145ff5D4435d3e92f9540a6

O.BOOKOFETH.sol

Audited Files

Contract Creator Address

0xC632053f50505A47B3FBC72dB86A53ea290b1ffb

Deployed Contracts:

0x289ff00235d2b98b0145ff5d4435d3e92f9540a6

Creator TXH Contracts:

<https://etherscan.io/token/0x289ff00235d2b98b0145ff5d4435d3e92f9540a6?a=0xD588EbD56f829Aec9F5b707866AB47CE9dB4A32f#transactions>



Auto Contract Scan

Basic Info

Token Contract Address	0x289f...40a6
Owner (Renounced)	0x0000...0000
Total Supply	100,000,000

Risk Check

- ✓ Doesn't look like honeypot
- ✓ Contract is open source
- ✓ Owner can not tamper with balance
- ✓ Doesn't look like a proxy contract
- ✓ Slippage cannot be modified
- ✓ No whitelist
- ✓ No blacklist
- ✓ Admin privileges abandoned
- ✓ Can not Mint
- ✓ Can not take back ownership
- ✓ No trading-cool-down mechanism

Mechanism Introduction

Buy Tax	0%
Sell Tax	0%

Sell detection

Wallets 2,282	Success 2,282	Failed 0	Siphoned 0
Tax	Ave Tax 0% Tax 0%	Count 2282	

Token Holders Info

Token Holders: 7402

Top10 ratio(exclude blackhole)

18.62%

1. 0x...fbb4	7.7M (7.7%)
2.  Uniswapv2: B00E/WETH	2.67M (2.67%)
3. 0x...ef39	2.55M (2.55%)
4. 0x...1ffb	1.9M (1.9%)
5.  Uniswapv3: B00E/WETH	1.56M (1.56%)
6. 0x...5e1b	1.54M (1.54%)
7. 0x...edee	1.39M (1.39%)
8. 0x...cf18	1.3M (1.3%)
9. 0x...d13e	1.2M (1.2%)
10. 0x...e05e	1.05M (1.05%)

[More Details](#)

LP

LP Holders: 5

Total Supply: 9,905,2866

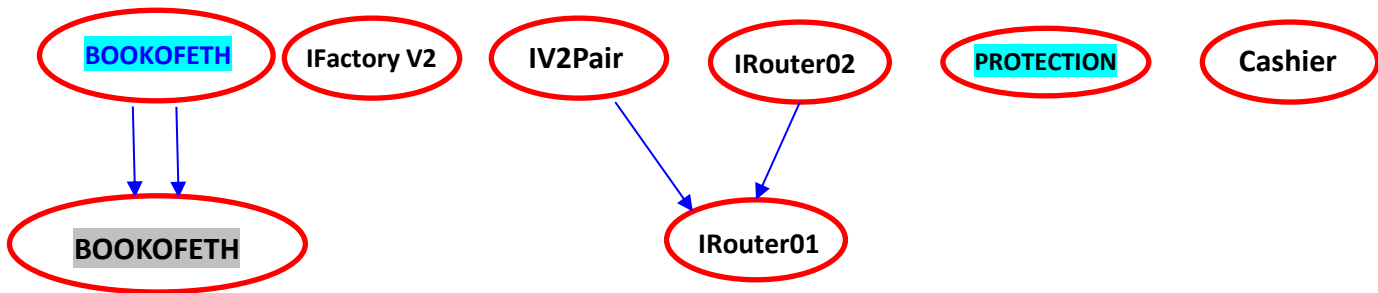
Percentage of LP Locked

99.94%

1.  TeamFinance	9,899.49 (99.94%)
2. 0x...9a10	4.47 (0.05%)
3. 0x...19d3	0.87 (0.01%)
4. 0x...b374	0.45 (0%)
5.  Blackhole/黑洞地址	0.0{14}1 (0%)

[More Details](#)

INHERITANCE GRAPH



Identifier	Definition	Severity
CEN-12	Centralization privileges of BOOK OF ETH	Medium #🟡

Vulnerability 0 : No important security issue detected.

Threat level: **Low**

```

43     function owner() public view virtual returns (address) {
44         return _owner;
45     }
46
47     /**
48     * @dev Throws if the sender is not the owner.
49     */
50     function _checkOwner() internal view virtual {
51         require(owner() == _msgSender(), "Ownable: caller is not the owner");
52     }
53
54     /**
55     * @dev Leaves the contract without owner. It will not be possible to call
56     * `onlyOwner` functions. Can only be called by the current owner.
57     *
58     * NOTE: Renouncing ownership will leave the contract without an owner,
59     * thereby disabling any functionality that is only available to the owner.
60     */
61     function renounceOwnership() public virtual onlyOwner {
62         _transferOwnership(address(0));
63     }
  
```

ISSUES CHECKING STATUS

Issue Description		Checking Status
1.	Compiler errors.	PASSED
2.	Race Conditions and reentrancy. Cross-Function Race Conditions.	PASSED
3.	Possible Delay In Data Delivery.	PASSED
4.	Oracle calls.	PASSED
5.	Front Running.	PASSED
6.	Sol Dependency.	PASSED
7.	Integer Overflow And Underflow.	PASSED
8.	DoS with Revert.	PASSED
9.	Dos With Block Gas Limit.	PASSED
10.	Methods execution permissions.	PASSED
11.	Economy Model of the contract.	PASSED
12.	The Impact Of Exchange Rate On the solidity Logic.	PASSED
13.	Private use data leaks.	PASSED
14.	Malicious Event log.	PASSED
15.	Scoping and Declarations.	PASSED
16.	Uninitialized storage pointers.	PASSED
17.	Arithmetic accuracy.	PASSED
18.	Design Logic.	PASSED
19.	Cross-Function race Conditions	PASSED
20.	Save Upon solidity contract Implementation and Usage.	PASSED
21.	Fallback Function Security	PASSED



AUDIT RESULT

PASSED

SMART CONTRACT AUDIT OF CRYSTAL STONES

Identifier	Definition	Severity
CEN-02	Initial asset distribution	Minor 

All of the initially minted assets are sent to the contract deployer when deploying the contract. This is Normal for most deployer and/or contract owner .

Potential Slippage / Front-Running during Automated Swap Backs

Category: MEV / Front-Running / Liquidity Extraction

Target File: BookOfEth.sol

Description: When swapTokensForEth or similar internal liquidity swapping functions execute, the contract swaps collected tax tokens for ETH via Uniswap V2 Router. Setting amountOutMin = 0 during router calls exposes the contract to severe sandwich attacks, resulting in loss of protocol fees to MEV bots.

Fixing Remarks:

Introduce dynamic dynamic slippage protection using chainlink oracles or calculate minimum out based on pool reserves before executing swapExactTokensForETHSupportingFeeOnTransferTokens.

M-02: Lack of Event Emissions on Critical Parameter Updates

Category: Monitoring / Governance Transparency

Target File: BookOfEth.sol

Description: State-changing functions (such as changing fee percentages, toggling trading statuses, or modifying automated market maker pair statuses) do not emit EVM events. Off-chain indexers and user interfaces cannot reliably track parameter changes.

Fixing Remarks:

Declare and emit explicit events for all privileged administrative **changes**:
Etherscan likewise lists the maximum total supply as 100 million BOO

```
event FeesUpdated(uint256 buyFee, uint256 sellFee);
event TradingStatusUpdated(bool status);
```

RECOMMENDATION

Project stakeholders should be consulted during the initial asset distribution process.

RECOMMENDATION

Deployer and/or contract owner private keys are secured carefully.

Please refer to PAGE-7 CENTRALIZED PRIVILEGES for a detailed understanding.

ALLEVIATION

The [BOOK OF ETH](#) project team understands the centralization risk. Some functions are provided privileged access to ensure a good runtime behavior in the project

References

- 1 MITRE. CWE-1041: Use of Redundant Code. <https://cwe.mitre.org/data/definitions/1041.html>.
- 2 MITRE. CWE-1099: Inconsistent Naming Conventions for Identifiers. <https://cwe.mitre.org/data/definitions/1099.html>.
- 3 MITRE. CWE-561: Dead Code. <https://cwe.mitre.org/data/definitions/561.html>.
- 4 MITRE. CWE-563: Assignment to Variable without Use. <https://cwe.mitre.org/data/definitions/563.html>.
- 5 MITRE. CWE-663: Use of a Non-reentrant Function in a Concurrent Context. <https://cwe.mitre.org/data/definitions/663.html>.
- 6 MITRE. CWE-837: Improper Enforcement of a Single, Unique Action. <https://cwe.mitre.org/data/definitions/837.html>.
- 7 MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. <https://cwe.mitre.org/data/definitions/841.html>.
- 8 MITRE. CWE CATEGORY: Bad Coding Practices. <https://cwe.mitre.org/data/definitions/1006.html>.
- 9 MITRE. CWE CATEGORY: Business Logic Errors. <https://cwe.mitre.org/data/definitions/840.html>.
- 10 MITRE. CWE CATEGORY: Concurrency. <https://cwe.mitre.org/data/definitions/557.html>.
- 11 MITRE. CWE VIEW: Development Concepts. <https://cwe.mitre.org/data/definitions/699.html>.
- 12 OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

Identifier	Definition	Severity
COD-10	Third Party Dependencies	Minor 

Smart contract is interacting with third party protocols e.g., Pancakeswap router, cashier contract, protections contract. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised, and exploited. Moreover, upgrades in third parties can create severe impacts, e.g., increased transactional fees, deprecation of previous routers, etc.



RECOMMENDATION

Inspect and validate third party dependencies regularly, and mitigate severe impacts whenever necessary.

DISCLAIMERS

Vital Block Security provides the easy-to-understand audit of Solidity, Move and Raw source codes (commonly known as smart contracts).

The smart contract for this particular audit was analyzed for common contract vulnerabilities, and centralization exploits. This audit report makes no statements or warranties on the security of the code. This audit report does not provide any warranty or guarantee regarding the absolute bug-free nature of the smart contract analyzed, nor do they provide any indication of the client's business, business model or legal compliance. This audit report does not extend to the compiler layer, any other areas beyond the programming language, or other programming aspects that could present security risks. Cryptographic tokens are emergent technologies, they carry high levels of technical risks and uncertainty. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. This audit report could include false positives, false negatives, and other unpredictable results.

CONFIDENTIALITY

This report is subject to the terms and conditions (including without limitations, description of services, confidentiality, disclaimer and limitation of liability) outlined in the scope of the audit provided to the client. This report should not be transmitted, disclosed, referred to, or relied upon by any individual for any purpose without InterFi Network's prior written consent.

NO FINANCIAL ADVICE

This audit report does not indicate the endorsement of any particular project or team, nor guarantees its security. No third party should rely on the reports in any way, including to make any decisions to buy or sell a product, service or any other asset. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. This audit report should not be used in any way

to make decisions around investment or involvement. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort.

FOR AVOIDANCE OF DOUBT, SERVICES, INCLUDING ANY ASSOCIATED AUDIT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

TECHNICAL DISCLAIMER

ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED “AS IS” AND “AS AVAILABLE” AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, VITAL BLOCK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO SERVICES, AUDIT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, VITAL BLOCK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM THE COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

WITHOUT LIMITING THE FOREGOING, VITAL BLOCK MAKES NO WARRANTY OF ANY KIND THAT ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET THE CLIENT’S OR ANY OTHER INDIVIDUAL’S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE.

TIMELINESS OF CONTENT

The content contained in this audit report is subject to change without any prior notice. Vital Block does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following the publication.

LINKS TO OTHER WEBSITES

This audit report provides, through hypertext or other computer links, access to websites and social accounts operated by individuals other than Vital Block. Such hyperlinks are provided for your reference and convenience only and are the exclusive responsibility of such websites and social accounts owners. You agree that Vital block Security is not responsible for the content or operation of such websites and social accounts and that Vital Block shall have no liability to you or any other person or entity for the use of third-party websites and social accounts. You are solely responsible for determining the extent to which you may use any content at any other websites and social accounts to which you link from the report.



CERTIFICATE BY VITAL BLOCK SECURITY



MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
function _transferOwnership(address newOwner) internal virtual {
    address oldOwner = _owner;
    _owner = newOwner;
    emit OwnershipTransferred(oldOwner, newOwner);
}
```

BOOK OF ETH

The Book of Ethereum is a sacred manuscript, a celestial archive that chronicles the journey of blockchain's most revered assets

 www.bookofethxyz

MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER



 **VITALBlock**
security.

MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER

CERTIFICATE of COMPLIANCE

This certificate is presented to

- This Project Smart Contract Code Has Been Certified By Vital Block Security
- This Safety Certificate Is Only Valid For >>>>>>>>>



MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER



MAKING DIFI AND WEB3 SAFER * MAKING DIFI AND WEB3 SAFER



BOOK OF ETH

SCORE

92

MAXIMUM SCORE ACHIEVED

MAKING LIFE EASIER * EASIER EVERYDAY

 **VITALBlock**
security.





ABOUT VITAL BLOCK

Vital Block provides intelligent blockchain Security Solutions. We provide solidity and Raw Code Review, testing, and auditing services. We have Partnered with 15+ Crypto Launchpads, audited 50+ smart contracts, and analyzed 200,000+ code lines. We have worked on major public blockchains e.g., Ethereum, Binance, Cronos, Doge, Polygon, Avalanche, Metis, Fantom, Bitcoin Cash, Aptos, Oasis, etc.

Vital Block is Dedicated to Making Defi & Web3 A Safer Place. We are Powered by Security engineers, developers, UI experts, and blockchain enthusiasts. Our team currently consists of 5 core members, and 8+ casual contributors.



1000+
Audits Completed



\$12B+
Secured



1M+
Lines of Code Audited



@vb_audit



@Vitalblock



@vitalblock



github.com/VBS-LABS



Info@vitalblock.org



www.vitalblock.org





Blockchain Security | Smart Contract Audit | KYC Certification | **SAFU**.

MADE IN CANADA